

Cahier de Charges

Étude et implémentation d'un système SCADA
(Supervisory Control And Data Acquisition)
basé sur des microcontrôleurs ESP32

Table des matières

Table des matières	2
Introduction	3
Problématique	3
Contexte	3
Méthodologie	3
Architecture du système	3
Technologies utilisées	4
Exigences fonctionnelles	4
Fonctions principales	4
Exigences non fonctionnelles	5
Planning prévisionnel	5
Ressources et budget	6
Conclusion	8
Importance du thème	8
Bibliographie	9

Introduction

Problématique

Comment concevoir et réaliser un système SCADA basé sur des microcontrôleurs ESP32 permettant la supervision et le contrôle d'un processus automatisé via un réseau local, tout en évaluant les performances comparées des protocoles de communication MQTT et HTTP en termes de latence, fiabilité et sécurité ?

Contexte

Les systèmes SCADA (Supervisory Control And Data Acquisition) constituent le cœur de la supervision industrielle moderne. Traditionnellement fondés sur des automates programmables et des réseaux propriétaires, ils évoluent aujourd'hui vers des architectures ouvertes intégrant les technologies de l'Internet des Objets Industriel (IIoT).

Le microcontrôleur ESP32, grâce à ses capacités Wi-Fi intégrées, son faible coût et sa richesse en interfaces (GPIO, ADC, I2C, SPI), représente une plateforme idéale pour expérimenter ces nouvelles architectures dans un contexte académique et industriel.

Ce projet s'inscrit dans ce contexte et vise à produire un système fonctionnel, documenté et évalué selon des critères techniques rigoureux.

Méthodologie

Architecture du système

Le système est organisé en trois niveaux hiérarchiques :

- Niveau terrain : nœuds ESP32 équipés de capteurs (température, humidité, pression) responsables de l'acquisition et de la transmission des données.

- Niveau communication : broker MQTT (Mosquitto) et API REST HTTP ~~assurant l'acheminement des données vers le serveur central via un réseau Wi-Fi local.~~
- Niveau supervision : serveur central hébergeant la base de données, le moteur de traitement et le tableau de bord de visualisation en temps réel.

Technologies utilisées

Le développement repose sur les outils et technologies suivants :

- Matériel : microcontrôleurs ESP32, capteurs DHT22, BMP280, capteurs de courant ACS712.
- Protocoles : MQTT v3.1.1 (broker Mosquitto) et HTTP/1.1 REST (Flask/FastAPI).
- Sécurité : chiffrement TLS 1.2, authentification par certificat et token.
- Stockage : InfluxDB pour les séries temporelles, SQLite en alternative légère.
- Supervision : Node-RED ou Grafana pour la visualisation en temps réel.
- Développement : PlatformIO, Arduino IDE, Python 3.x.
- Versionnage : Git / GitHub pour le suivi du code source.

Exigences fonctionnelles

Fonctions principales

- Acquisition périodique des données capteurs (intervalle paramétrable).
- Transmission des données via MQTT et HTTP de manière configurable.
- Affichage en temps réel des valeurs sur un tableau de bord web.

- Génération d'alarmes sur dépassement de seuils définis.
- Enregistrement horodaté de toutes les mesures en base de données.
- Export des données historiques aux formats CSV et JSON.
- Sécurisation des échanges par chiffrement TLS et authentification.
- Mesure automatisée de la latence, du taux de perte et du débit réseau.

Exigences non fonctionnelles

- Latence MQTT inférieure à 100 ms en conditions réseau locales normales.
- Latence HTTP inférieure à 300 ms pour une requête POST simple.
- Disponibilité supérieure à 99 % sur la durée des tests.
- Support d'au moins 5 nœuds ESP32 simultanément.
- Code modulaire, documenté et versionné sur Git.
- Consommation du nœud en veille inférieure à 10 mA.

Planning prévisionnel

Phase 1 – État de l'art (3 semaines)

Bibliographie SCADA, IIoT, ESP32, protocoles de communication.

Phase 2 – Conception (2 semaines)

Architecture système, choix technologiques, schémas matériels.

Phase 3 – Développement (5 semaines)

Firmware ESP32, broker MQTT, API HTTP, base de données.

Phase 4 – Supervision (3 semaines)

Tableau de bord temps réel, alertes, export de données.

Phase 5 – Sécurité (2 semaines)

Intégration TLS, authentification, tests de vulnérabilité.

Phase 6 – Évaluation (3 semaines)

Mesures latence et fiabilité, analyse comparative MQTT vs HTTP.

Phase 7 – Rédaction (4 semaines)

Rédaction du mémoire, figures, préparation de la soutenance.

Durée totale estimée : 22 semaines (environ 5 mois).

Ressources et budget

Les ressources nécessaires à la réalisation du projet sont les suivantes :

- 5 cartes ESP32 DevKit — environ 25 000 FCFA.
- Capteurs divers (température, humidité, pression) — environ 15 000 FCFA.
- 1 Raspberry Pi 4 (serveur central) — environ 45 000 FCFA.
- Composants électroniques divers et PCB — environ 10 000 FCFA.
- Logiciels : entièrement open-source (coût nul).

- Hébergement cloud optionnel — environ 5 000 FCFA.

Budget total estimé : environ 100 000 FCFA.

Conclusion

Importance du thème

Ce projet de système SCADA basé sur des microcontrôleurs ESP32 et les technologies IIoT permet de démontrer l'importance de l'intégration entre l'électronique embarquée et les réseaux informatiques dans les systèmes industriels modernes.

Sur le plan académique, il offre une opportunité concrète d'explorer des problématiques actuelles : performance des protocoles de communication, sécurité des systèmes embarqués connectés, et fiabilité des architectures distribuées à faible coût.

Sur le plan industriel, les résultats de ce projet pourront servir de référence pour évaluer la pertinence de solutions IIoT légères comme alternative aux infrastructures SCADA classiques dans des environnements de supervision à taille réduite.

Bibliographie

(Feasibility Evaluation of a Microcontroller-Based SCADA Prototype as an Operational Traction Substation Monitoring System. Journal of Railway Transportation and Technology.)

(Low-Cost, Open-Source, Emoncms-Based SCADA System for a Large Grid-Connected PV System.)

(MQTT Version 3.1.1 – OASIS Standard, octobre 2014. <https://docs.oasis-open.org/mqtt/>)

(Espressif Systems. ESP32 Technical Reference Manual. Version 5.1, 2023.)

(Mosquitto – An open source MQTT broker. Eclipse Foundation. <https://mosquitto.org>)

(Grafana Labs. Grafana Documentation. <https://grafana.com/docs/>)

(InfluxData. InfluxDB Documentation. <https://docs.influxdata.com/>)

(Node-RED. Documentation officielle. <https://nodered.org/docs/>)